



Getting Started with DTracing MySQL

...to understand the runtime behaviour of the RDBMS better.

DTrace is a dynamic tracing facility built into the Solaris and Open Solaris operating systems and can be used by systems administrators and developers to observe the runtime behaviour of user-level programs and of the OS itself. On one hand, DTrace can be used to identify potential bottlenecks in the running processes on a production system, while on the other hand it can help you understand the runtime behaviour of an external program such as MySQL better.

Originally available on Solaris, DTrace has now been ported to Mac OSX, FreeBSD and an experimental Linux port is also available. In this article, I shall use the OpenSolaris 2008.11 release to demonstrate how it works.

Some concepts first

The DTrace architecture [check the *References*] gives you a good look at the various

components of the DTrace framework. The graphic in Figure 1 (reproduced from the DTrace how-to at www.sun.com/software/solaris/howtoguides/dtrac-howto.jsp) illustrates the DTrace framework and its various components. Note that 'probes', about which we will learn more shortly (and not shown in the figure) can be best visualised as sensors available to be probed by the DTrace consumers in user space.

We shall now learn the basic DTrace concepts that will help us during some serious playing around with DTrace and MySQL.

Probes, providers and consumers

DTrace dynamically modifies the operating system kernel and user processes to record data at locations of interest (or instrumentation points) called probes. The probe is user specified, and its specificity and description determine the benefit derived